

Flutter Day 1 - Delegate Handbook

Foundations & Dart · MO Integrations · Flutter Mobile Application Development

How to use this document. Open it in VS Code (or any editor) alongside the course so you can copy blocks straight out of it. Every code block is complete and runnable as it stands - nothing is a fragment. Windows commands come first because that is what the room runs; macOS follows each one.

Course project: `quote_app` - a white-label motor-insurance quoting app. Everything you build today is used again on Days 2-5. Nothing is throwaway.

Part 0 · Before we start

You need	Notes
Windows 10/11 (or macOS)	16 GB RAM recommended, 8 GB minimum, 30 GB free disk
Flutter SDK, stable channel	Part 1 below
Android Studio	For the Android SDK and the emulator - even though we code in VS Code
VS Code + Flutter extension	Our editor for the week
An Android emulator or a physical Android phone	Either is fine
A browser	dartpad.dev - Labs 1.2 and 1.3 run here, no install needed

If your install is not finished, do not panic. Labs 1.2 and 1.3 run entirely in the browser. You can catch up on the SDK during those labs.

Do this before you arrive, if you possibly can

Your very first Flutter build downloads about **4 GB** - a 2.8 GB Android NDK, the SDK platform, and the Gradle distribution - and takes seven to eight minutes. Twenty of us doing that at once on the venue network will not go well.

Once your install is done, run this at your own desk:

```
flutter create precourse_check
```

```
cd precourse_check
```

```
flutter build apk --debug
```

Wait for the line that says it built `app-debug.apk`, then delete the folder. Every build after that is seconds instead of minutes, and you have proved your setup works before Day 1 starts.

Part 1 • Install the Flutter SDK

Windows (PowerShell)

Two rules: **no spaces in the path**, and **not inside Program Files**.

```
# 1. Download the stable Windows zip from
#   https://docs.flutter.dev/get-started/install/windows
#   Extract it to C:\src\flutter (so you end up with C:\src\flutter\bin)
Expand-Archive "$HOME\Downloads\flutter_windows_*-stable.zip" C:\src
```

```
# 2. Add to your USER Path (this appends without duplicating machine entries)
$u = [Environment]::GetEnvironmentVariable("Path", "User")
[Environment]::SetEnvironmentVariable("Path", "$u;C:\src\flutter\bin", "User")
```

```
# 3. CLOSE this terminal, open a NEW PowerShell, then:
flutter --version
```

```
# 4. Two Windows housekeeping items
git config --system core.longpaths true
# Add C:\src\flutter and your projects folder to your antivirus exclusions,
# or Gradle builds will be 3-5x slower.
```

macOS (zsh)

```
# 1. Download the stable macOS zip (Apple Silicon or Intel) from
#   https://docs.flutter.dev/get-started/install/macos
mkdir -p ~/development
cd ~/development
unzip ~/Downloads/flutter_macos_*-stable.zip
```

```
# 2. PATH
echo 'export PATH="$HOME/development/flutter/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

```
# 3. Verify
flutter --version
```

```
# 4. Only if you want to build for iOS (not needed for this course)
xcode-select --install
sudo gem install cocoapods
```

Part 2 • Android toolchain

Install **Android Studio** from <https://developer.android.com/studio> and run the first-run wizard with the **Standard** install.

Then **Android Studio** → **More Actions** → **SDK Manager**:

SDK Platforms tab - tick the latest stable API.

SDK Tools tab - tick: - Android SDK Build-Tools - Android SDK Command-line Tools (latest) ← *the one people miss* - Android Emulator - Android SDK Platform-Tools - **Windows only:** Android Emulator Hypervisor Driver

Then accept the licences:

```
flutter doctor --android-licenses
# press y at every prompt
```

Windows: emulator acceleration

The emulator needs hardware virtualisation. Either the **Android Emulator Hypervisor Driver** above, or **Windows Hypervisor Platform**:

Control Panel → Programs → Turn Windows features on or off → tick **Windows Hypervisor Platform** → reboot

If your BIOS has virtualisation disabled, or company policy blocks Hyper-V, the emulator will not boot. Use a **physical phone** (Part 4) or `flutter run -d chrome`. This is normal and everything today still works.

Part 3 • flutter doctor

```
flutter doctor
flutter doctor -v          # verbose: shows the paths it is using
```

You need green ticks for **Flutter**, **Android toolchain** and **one IDE**. These warnings are fine and can be ignored today:

- `[!]` Visual Studio - only needed for Windows desktop apps
- `[!]` Chrome - only needed for web

PowerShell 5.1 has no `&&`. Windows 10/11 opens Windows PowerShell 5.1 by default, and chaining commands with `&&` gives "*The token '&&' is not a valid statement separator in this version*". Every command in this handbook is one line at a time. Run them one at a time.

Common fixes

Message	Fix
<code>flutter</code> is not recognised	You did not open a new terminal after changing Path
<code>cmdline-tools</code> component is missing	SDK Manager → SDK Tools → tick <i>Android SDK Command-line Tools (latest)</i>
Android license status unknown and <code>--licenses</code> replies "no longer needed"	Expected on current tools - not your machine. Android replaced <code>sdkmanager</code> with the new <code>android</code> CLI, and Flutter's licence check has not caught up (flutter #191487). Your licences <i>are</i> accepted. Verify with the build test below and carry on.
Android license status unknown on older command-line tools	<code>flutter doctor --android-licenses</code> then y to everything
Multiple adb binaries found	You have platform-tools twice. Keep Android Studio's and remove the other. macOS: <code>brew uninstall --cask android-platform-tools</code> . Windows: take the standalone platform-tools folder off your PATH.
Unable to find git in your PATH	Install Git for Windows, tick "Git from the command line", reopen terminal
Android Studio not installed (but it is)	<code>flutter config --android-studio-dir "C:\Program Files\Android\Android Studio"</code>
Unsupported class file major version during build	<code>flutter config --jdk-dir "C:\Program Files\Android\Android Studio\jbr"</code>

```
# Useful anytime
flutter upgrade
flutter channel stable
flutter config --android-sdk "$env:LOCALAPPDATA\Android\Sdk"
```

The only test that really matters

`flutter doctor` can be wrong. A build cannot. If doctor shows the licence X but this succeeds, your Android setup is fine:

```
flutter create sanity_check
cd sanity_check
flutter build apk --debug
```

A `Built build\app\outputs\flutter-apk\app-debug.apk` line means you are ready. Delete the folder afterwards.

Part 4 • Create an emulator, or use a phone

Emulator

Android Studio → Device Manager → Create Device → Pixel 8 → the latest stable API (x86_64 on Windows, arm64-v8a on Apple Silicon) → Finish → ►

Start it once now - the first boot takes 1-3 minutes.

```
flutter emulators                # list AVDs
flutter emulators --launch pixel8 # start one
flutter devices                  # what flutter run can target
```

Physical Android phone (faster, and the fallback if the emulator fails)

1. On the phone: **Settings → About phone → tap Build number 7 times**
2. **Settings → Developer options → USB debugging ON**
3. Plug in with a **data** cable (many charging cables have no data lines)
4. Accept the "Allow USB debugging" prompt on the phone

```
adb devices          # the phone should be listed
flutter devices
flutter run
```

If `adb devices` is empty on Windows, you need the OEM USB driver: Google/Pixel → *Google USB Driver* in SDK Manager; Samsung → Smart Switch; Huawei → HiSuite.

```
# adb lives here if it is not on your Path
& "$env:LOCALAPPDATA\Android\Sdk\platform-tools\adb.exe" devices
```

Part 5 • VS Code setup

Install the **Flutter** extension (it pulls in Dart automatically).

Shortcuts you will use all week:

Action	Windows	macOS
Command Palette	Ctrl+Shift+P	Cmd+Shift+P
Quick fix / refactor	Ctrl+.	Cmd+.
Start debugging	F5	F5
Format document	Shift+Alt+F	Shift+Option+F

Optional project settings - create `.vscode/settings.json` :

```
{
  "editor.formatOnSave": true,
  "dart.lineLength": 100,
  "dart.previewFlutterUiGuides": true,
  "dart.flutterHotReloadOnSave": "all",
  "[dart]": {
    "editor.rulers": [100],
    "editor.selectionHighlight": false,
    "editor.tabCompletion": "onlySnippets",
    "editor.wordBasedSuggestions": "off"
  }
}
```

Snippets: type `stless` or `stful` then Tab.

LAB 1.1 • Toolchain verified • 25 min

Goal: the template app runs on your emulator or phone with working hot reload.

```
# 1. Confirm the toolchain
flutter doctor
```

```
# 2. Create the project (--org sets the Android package / iOS bundle id - do it now, changing
it later is painful)
flutter create --org za.co.moint quote_app
cd quote_app
```

```
# 3. Start your emulator (or plug in the phone), then confirm Flutter sees it
flutter devices
```

```
# 4. Run it. If you did the pre-course build this is quick. If not, expect 7-8 minutes and a
~4 GB download the first time.
flutter run
```

5. Open `lib/main.dart`, change the AppBar title text, save, then press `r` in the terminal. The title changes and the counter keeps its value - that is **hot reload**.

6. Press `R` (capital). The app restarts and the counter resets to 0 - that is **hot restart**.

7. Press `q` to quit. Then open the folder in VS Code and press `F5` to run under the debugger.

```
# 8. Commit
git init
git add .
git commit -m "Day 1: template app"
```

Hot reload keys (in the `flutter run` terminal)

Key	Does
<code>r</code>	Hot reload - inject changed code, keep state
<code>R</code>	Hot restart - re-run <code>main()</code> , lose state
<code>p</code>	Toggle debug paint (draws boxes round every widget)
<code>o</code>	Toggle Android / iOS rendering
<code>q</code>	Quit

Hot reload does **not** pick up changes to `main()`, global initialisers, enum values, or `const` values - press `R` for those. If you add a package with native code, stop and `flutter run` again.

Stretch: run it in the browser with `flutter run -d chrome`, and open `android/app/build.gradle.kts` to find `applicationId` - we change that on Day 5.

Part 6 • Dart primer - copy along in DartPad

Open <https://dartpad.dev> → New Pad → Dart. Paste any block below into the pad (replacing what is there) and press Run.

Variables

```
void main() {
  var count = 0;           // int, inferred, reassignable
  count = 5;
  // count = 'five';       // compile error: String is not int

  final createdAt = DateTime.now(); // set once, at runtime (= JS const)
  const vatRate = 0.15;           // compile-time constant
  const brand = 'Alpha Insure';

  double premium = 1450.0;        // explicit type where it helps the reader
  List<String> makes = ['Toyota', 'VW'];

  print('$brand $count $createdAt $vatRate $premium $makes');
}
```

Null safety

```
void main() {
  String name = 'Sive';
  // name = null;           // compile error

  String? nickname;         // may be null
  print(nickname?.length);  // null
  print(nickname ?? 'none'); // 'none'
  nickname ??= 'S';         // assign only if null
  print(nickname!.length);  // != "trust me, not null" - throws if wrong

  String? maybe = DateTime.now().hour > 0 ? 'value' : null;
  if (maybe != null) {
    print(maybe.length);    // promoted to String inside the if
  }
}
```

Fields do not promote. This is the one that catches everybody:

```

class Driver {
  String? nickname;

  void greet() {
    // if (nickname != null) print(nickname.length); // ERROR on a field
    final n = nickname; // copy to a local
    if (n != null) print(n.length); // now it promotes
    print(nickname?.length ?? 0); // or just be null-aware
  }
}

void main() => Driver()..nickname = 'Sive'..greet();

```

Functions

```

double premium(double base, {required int age, bool comprehensive = true}) {
  final loading = age < 25 ? 1.5 : 1.0;
  return comprehensive ? base * 1.4 * loading : base * loading;
}

void main() {
  print(premium(1000, age: 34)); // 1400.0
  print(premium(1000, age: 22, comprehensive: false)); // 1500.0

  final int Function(int) twice = (x) => x * 2;
  print(twice(21));
}

```

Named parameters in `{}` are how every widget constructor works. `required` makes one mandatory.

Classes

```
class Quote {
    final String id;
    final double premium;
    final String currency;

    const Quote({required this.id, required this.premium, this.currency = 'ZAR'});

    String get display => '$currency ${premium.toStringAsFixed(2)}';

    factory Quote.fromJson(Map<String, dynamic> j) => Quote(
        id: j['id'] as String,
        premium: (j['premium'] as num).toDouble(),
        currency: j['currency'] as String? ?? 'ZAR',
    );

    Map<String, dynamic> toJson() => {'id': id, 'premium': premium, 'currency': currency};
}

void main() {
    const q = Quote(id: 'q1', premium: 1450);
    print(q.display);
    print(q.toJson());
    print(Quote.fromJson({'id': 'q2', 'premium': 900}).display);
}
```

Collections

```
void main() {
    final makes = <String>['Toyota', 'VW', 'BMW'];
    final byMake = {'VW': 1200.0, 'BMW': 2100.0};
    const showLuxury = true;

    final list = [
        'Toyota',
        if (showLuxury) 'BMW', // collection-if
        for (final m in makes) m.toUpperCase(), // collection-for
        ...makes, // spread
    ];
    print(list);

    final cheap = byMake.entries
        .where((e) => e.value < 2000)
        .map((e) => e.key)
        .toList(); // <-- .toList() matters
    print(cheap);

    print(byMake.values.fold<double>(0.0, (a, b) => a + b));
}
```

`map()` and `where()` return a lazy `Iterable`. Call `.toList()` when you need a `List` - forgetting it is the number one beginner error.

Switch expressions and sealed classes

```
enum Cover { thirdParty, thirdPartyFireTheft, comprehensive }

double factor(Cover c) => switch (c) {
    Cover.thirdParty => 0.6,
    Cover.thirdPartyFireTheft => 0.8,
    Cover.comprehensive => 1.0,
};

String band(int age) => switch (age) {
    < 18 => 'not eligible',
    >= 18 && < 25 => 'young driver',
    _ => 'standard',
};

void main() {
    for (final c in Cover.values) print('$c → ${factor(c)}');
    print(band(22));
}
```

Async

```
Future<String> fetchQuote(String id) async {
    await Future.delayed(const Duration(milliseconds: 500));
    if (id.isEmpty) throw Exception('no id');
    return 'quote-$id';
}

Stream<int> countdown(int from) async* {
    for (var i = from; i >= 0; i--) {
        await Future.delayed(const Duration(milliseconds: 300));
        yield i;
    }
}

void main() async {
    try {
        print(await fetchQuote('q1'));
    } catch (e, st) {
        print('$e\n$st');
    }

    await for (final n in countdown(3)) print(n);

    final sw = Stopwatch()..start();
    await Future.wait([fetchQuote('a'), fetchQuote('b'), fetchQuote('c')]);
    print('3 in parallel took ${sw.elapsedMilliseconds} ms'); // ~500, not 1500
}
```

`Future` = `Promise` / `Task<T>`. `Stream` = `Observable` / `IAsyncEnumerable<T>`.

Mixins and extensions

```
mixin Loggable {
  void log(String m) => print('[$runtimeType] $m');
}

extension Money on double {
  String get rands => 'R ${toStringAsFixed(2)}';
}

class QuoteService with Loggable {
  void quote() => log('quoting...');
}

void main() {
  QuoteService().quote();
  print(1450.0.rands);           // R 1450.00
}
```

LAB 1.2 • Quote domain model • 35 min

Goal: a null-safe, immutable domain model, exercised from `main()`. Runs entirely in **dartpad.dev** - no SDK needed.

Ready-to-paste versions of every code block today are in the course repo under `dartpad/` - open the file, select all, paste into dartpad.dev, Run.

Starter - paste this into DartPad and fill in the TODOs

```
// ----- 1. Cover -----
enum Cover {
  thirdParty, thirdPartyFireTheft, comprehensive;

  // TODO: double get factor => switch (this) { ... 0.6 / 0.8 / 1.0 ... };
}

// ----- 2. QuoteRequest -----
// TODO: fields make (String), model (String?), year (int), driverAge (int), cover (Cover)
//       const constructor with required named params
//       copyWith

// ----- 3. Quote -----
// TODO: id (String), premium (double), currency defaulting to 'ZAR'
//       a `display` getter, fromJson / toJson

// ----- 4. Premium calculation -----
// TODO: double calculatePremium(QuoteRequest r)
//       base 1000; x1.5 if driverAge < 25; x1.2 if year < 2015; x cover.factor
//       round to 2 decimals

// ----- 5. Sealed state -----
// TODO: sealed class QuoteState with Idle / Loading / Loaded(quote) / Failed(message)
//       String describe(QuoteState s) using an exhaustive switch expression

void main() {
  // TODO: build three requests with copyWith, print each premium
  // TODO: print describe() for all four states
}
```

Solution - complete and runnable

```
enum Cover {
  thirdParty, thirdPartyFireTheft, comprehensive;
  double get factor => switch (this) {
    Cover.thirdParty => 0.6,
    Cover.thirdPartyFireTheft => 0.8,
    Cover.comprehensive => 1.0,
  };
}

class QuoteRequest {
  final String make;
  final String? model;
  final int year;
  final int driverAge;
  final Cover cover;

  const QuoteRequest({
    required this.make,
    this.model,
    required this.year,
    required this.driverAge,
    required this.cover,
  });

  QuoteRequest copyWith({String? make, String? model, int? year, int? driverAge, Cover?
cover}) =>
    QuoteRequest(
      make: make ?? this.make,
      model: model ?? this.model,
      year: year ?? this.year,
      driverAge: driverAge ?? this.driverAge,
      cover: cover ?? this.cover,
    );

  @override
  bool operator ==(Object o) =>
    o is QuoteRequest &&
    o.make == make &&
    o.model == model &&
    o.year == year &&
    o.driverAge == driverAge &&
    o.cover == cover;

  @override
  int get hashCode => Object.hash(make, model, year, driverAge, cover);
}

extension Money on double {
  String get rands => 'R ${toStringAsFixed(2)}';
}

class Quote {
  final String id;
```

```

final double premium;
final String currency;

const Quote({required this.id, required this.premium, this.currency = 'ZAR'});

String get display => '$currency ${premium.toStringAsFixed(2)}';

factory Quote.fromJson(Map<String, dynamic> j) => Quote(
  id: j['id'] as String,
  premium: (j['premium'] as num).toDouble(),
  currency: j['currency'] as String? ?? 'ZAR',
);

Map<String, dynamic> toJson() => {'id': id, 'premium': premium, 'currency': currency};
}

double calculatePremium(QuoteRequest r) {
  var p = 1000.0;
  if (r.driverAge < 25) p *= 1.5;
  if (r.year < 2015) p *= 1.2;
  p *= r.cover.factor;
  return (p * 100).roundToDouble() / 100;
}

sealed class QuoteState {
  const QuoteState();
}

class QuoteIdle extends QuoteState {
  const QuoteIdle();
}

class QuoteLoading extends QuoteState {
  const QuoteLoading();
}

class QuoteLoaded extends QuoteState {
  const QuoteLoaded(this.quote);
  final Quote quote;
}

class QuoteFailed extends QuoteState {
  const QuoteFailed(this.message);
  final String message;
}

String describe(QuoteState s) => switch (s) {
  QuoteIdle() => 'Fill in the form',
  QuoteLoading() => 'Calculating...',
  QuoteLoaded(:final quote) => 'Premium ${quote.display}',
  QuoteFailed(:final message) => 'Error: $message',
};

void main() {
  const base = QuoteRequest(make: 'VW', year: 2020, driverAge: 30, cover:

```

```

Cover.comprehensive);
final young = base.copyWith(driverAge: 22);
final old = base.copyWith(year: 2012, cover: Cover.thirdParty);

for (final r in [base, young, old]) {
  print('${r.make} ${r.year} → ${calculatePremium(r).rands}');
}

for (final s in [
  const QuoteIdle(),
  const QuoteLoading(),
  QuoteLoaded(Quote(id: 'q1', premium: calculatePremium(base))),
  const QuoteFailed('too old'),
]) {
  print(describe(s));
}
}

```

Expected output

```

VW 2020 → R 1000.00
VW 2020 → R 1500.00
VW 2012 → R 720.00
Fill in the form
Calculating...
Premium ZAR 1000.00
Error: too old

```

Stretch 1. Prove `r1.copyWith() == r1` now that `==` is overridden. 2. Add `QuoteExpired` to the sealed class and watch `describe()` refuse to compile until you handle it. That is the whole point of sealed classes. 3. Use the `Money` extension inside `Quote.display`.

Keep this pad open - Lab 1.3 builds on it.

LAB 1.3 • Async quote service • 25 min

Goal: a fake service with realistic latency, failure, timeout, and a `Stream` of states.

Add `import 'dart:async';` at the top of your Lab 1.2 pad (needed for `TimeoutException`), then add the code below and replace `main()`.

```

import 'dart:async';

// ... keep everything from Lab 1.2 above this line ...

abstract interface class QuoteService {
  Future<Quote> getQuote(QuoteRequest r);
}

class FakeQuoteService implements QuoteService {
  @override
  Future<Quote> getQuote(QuoteRequest r) async {
    await Future.delayed(const Duration(milliseconds: 1500));
    if (r.year < 2000) throw Exception('Vehicle too old to insure');
    return Quote(id: 'q- $\{r.hashCode\}$ ', premium: calculatePremium(r));
  }
}

Future<void> runOnce(QuoteService s, QuoteRequest r) async {
  try {
    final q = await s.getQuote(r).timeout(const Duration(seconds: 3));
    print('OK  $\{q.display\}$ ');
  } on TimeoutException {
    print('Timed out');
  } catch (e) {
    print('Failed:  $\{e\}$ ');
  }
}

Stream<QuoteState> quoteStates(QuoteService s, QuoteRequest r) async* {
  yield const QuoteLoading();
  try {
    yield QuoteLoaded(await s.getQuote(r));
  } catch (e) {
    yield QuoteFailed(e.toString());
  }
}

void main() async {
  final svc = FakeQuoteService();
  const ok = QuoteRequest(make: 'VW', year: 2020, driverAge: 30, cover: Cover.comprehensive);

  await runOnce(svc, ok);

  await for (final st in quoteStates(svc, ok)) {
    print(describe(st));
  }
  await for (final st in quoteStates(svc, ok.copyWith(year: 1998))) {
    print(describe(st));
  }

  final sw = Stopwatch()..start();
  await Future.wait([svc.getQuote(ok), svc.getQuote(ok), svc.getQuote(ok)]);
  print('3 parallel quotes in  $\{sw.elapsedMilliseconds\}$  ms'); // ~1500, not 4500
}

```


The last line is the point of `Future.wait`: three 1.5-second calls finish in about 1.5 seconds, not 4.5.

Stretch 1. Make `FakeQuoteService` fail randomly 30% of the time and add one retry with back-off. 2. Add `.distinct()` to the state stream and see what changes.

LAB 1.4 • Quote app skeleton • 40 min

Goal: the app runs on your emulator with the Lab 1.2 model inside it and one stateful screen.

1. Folder structure

```
# from inside quote_app
mkdir lib\core\theme, lib\core\routing, lib\core\network, lib\core\widgets
mkdir lib\features\onboarding\presentation
mkdir lib\features\quote\domain, lib\features\quote\data, lib\features\quote\presentation
```

```
# macOS / Linux
mkdir -p lib/core/{theme,routing,network,widgets}
mkdir -p lib/features/onboarding/presentation
mkdir -p lib/features/quote/{domain,data,presentation}
```

2. `lib/features/quote/domain/quote_model.dart`

Paste your **entire Lab 1.2 solution** into this file, then delete its `main()`. Add this import at the top - nothing else is needed:

```
// lib/features/quote/domain/quote_model.dart
// (paste the Lab 1.2 solution here, minus main())
```

3. `lib/main.dart`

```
import 'package:flutter/material.dart';
import 'package:quote_app/app.dart';

void main() => runApp(const QuoteApp());
```

4. lib/app.dart

```
import 'package:flutter/material.dart';
import 'package:quote_app/features/quote/presentation/capture_screen.dart';

class QuoteApp extends StatelessWidget {
  const QuoteApp({super.key});

  @override
  Widget build(BuildContext context) => MaterialApp(
    title: 'Quote App',
    theme: ThemeData(
      colorSchemeSeed: const Color(0xFF0B2545),
      useMaterial3: true,
    ),
    home: const CaptureScreen(),
  );
}
```

5. lib/features/quote/presentation/capture_screen.dart

```
import 'package:flutter/material.dart';
import 'package:quote_app/features/quote/domain/quote_model.dart';

class CaptureScreen extends StatefulWidget {
  const CaptureScreen({super.key});

  @override
  State<CaptureScreen> createState() => _CaptureScreenState();
}

class _CaptureScreenState extends State<CaptureScreen> {
  Cover _cover = Cover.comprehensive;
  double? _premium;

  void _calculate() {
    final r = QuoteRequest(
      make: 'VW',
      year: 2020,
      driverAge: 30,
      cover: _cover,
    );
    setState(() => _premium = calculatePremium(r));
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Get a quote')),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        children: [
          SegmentedButton<Cover>(
            segments: [
              for (final c in Cover.values)
                ButtonSegment(value: c, label: Text(c.name)),
            ],
            selected: {_cover},
            onSelectionChanged: (s) => setState(() => _cover = s.first),
          ),
          const SizedBox(height: 24),
          FilledButton(
            onPressed: _calculate,
            child: const Text('Calculate'),
          ),
          const SizedBox(height: 24),
          if (_premium != null) PremiumBadge(amount: _premium!),
        ],
      ),
    ),
  );
}

class PremiumBadge extends StatelessWidget {
```

```
const PremiumBadge({super.key, required this.amount});
final double amount;

@override
Widget build(BuildContext context) => Text(
  amount.rands,
  style: Theme.of(context).textTheme.headlineMedium,
);
}
```

6. Delete the generated test

`flutter create` wrote `test/widget_test.dart`, which still refers to `MyApp`. You just replaced that with `QuoteApp`, so `flutter analyze` will fail on it. `dart fix --apply` will not repair it. Delete it - we write a real one on Day 5.

```
del test\widget_test.dart
```

macOS:

```
rm test/widget_test.dart
```

7. Run it

```
flutter run
```

Tap the segments, tap Calculate, watch the premium change.

8. Try the Extract Widget refactor

Put your cursor on the `Text(...)` inside `PremiumBadge`, press `Ctrl+.` (`Cmd+.`), and look at the refactor menu - **Wrap with Padding / Center / Column, Extract Widget**. You will use this constantly.

9. Open the Widget Inspector

`Ctrl+Shift+P` → **Flutter: Open DevTools** → Widget Inspector → turn on **Select Widget Mode** → tap the premium text on the emulator. It jumps to your source line.

```
flutter analyze          # must be clean
dart fix --apply         # applies suggested fixes (adds missing const, etc.)
git add . && git commit -m "Day 1: skeleton"
```

Stretch 1. Make Calculate async: `await FakeQuoteService().getQuote(...)` with a `CircularProgressIndicator` while it runs. Add `if (!mounted) return;` after the await. 2. `flutter pub add intl`, then format with `NumberFormat.currency(locale: 'en_ZA', symbol: 'R ').format(premium)`. 3. Add a second screen and `Navigator.push` to it - a preview of Day 4.

Part 7 • Cheat sheets

Flutter CLI

Command	Does
<code>flutter create --org za.co.moint quote_app</code>	New project with the right package id
<code>flutter run</code> • <code>flutter run -d chrome</code>	Build and run
<code>flutter pub add dio</code>	Add a dependency
<code>flutter pub add --dev mocktail</code>	Add a dev dependency
<code>flutter pub get</code>	Install from pubspec
<code>flutter pub outdated</code>	What is newer
<code>flutter analyze</code>	Static analysis
<code>flutter test</code>	Run tests
<code>flutter clean</code>	Delete build output - first thing to try when a build misbehaves
<code>dart format .</code>	Format everything
<code>dart fix --apply</code>	Apply automated fixes

Coming from TypeScript, C# or Java

Concept	TypeScript	C#	Dart
Immutable binding	<code>const x = 1</code>	<code>readonly</code>	<code>final x = 1;</code>
Compile-time constant	-	<code>const</code>	<code>const x = 1;</code>
Nullable	<code>string null</code>	<code>string?</code>	<code>String?</code> (enforced at runtime too)
Named args	object destructuring	named arguments	<code>f({required int age})</code>
Interfaces	<code>interface</code>	<code>interface</code>	any class: <code>implements</code>
Mixins	-	-	<code>mixin M {}</code> • <code>class A with M</code>
Tuples / records	tuple types	records	<code>(double, String)</code>
Discriminated union	<code>union + kind</code>	abstract records	<code>sealed class + switch</code>
Async	<code>Promise<T></code>	<code>Task<T></code>	<code>Future<T></code>
Streams	Observable	<code>IEnumerable<T></code>	<code>Stream<T></code>
Packages	npm / package.json	NuGet / .csproj	pub / pubspec.yaml

Coming from React Native

React Native	Flutter
Component	Widget
<code>useState</code>	<code>StatefulWidget</code> + <code>setState</code>
Props	named constructor params with <code>required</code>
JSX children	<code>child:</code> / <code>children:</code> <code>[...]</code>
<code>View</code> + flex	<code>Row</code> / <code>Column</code> + <code>Expanded</code>
<code>FlatList</code>	<code>ListView.builder</code>
<code>TextInput</code>	<code>TextField</code> / <code>TextFormField</code>
<code>useEffect</code> mount/cleanup	<code>initState</code> / <code>dispose</code>
Context / Zustand	Riverpod (Day 3)
Expo Router	<code>go_router</code> (Day 4)
Fast Refresh	Hot reload (<code>r</code>)

Five Dart habits that catch newcomers

Habit	What happens	Do this instead
Forgetting <code>.toList()</code>	<code>map</code> / <code>where</code> give a lazy <code>Iterable</code> ; type error	Chain <code>.toList()</code>
Using <code>!</code> to silence the compiler	Runtime null crash later	<code>?.</code> , <code>??</code> , early return, <code>required</code>
Mutating state objects	Widgets do not rebuild	New object + <code>copyWith</code>
No <code>==</code> on your own classes	Two equal objects compare as different	Override <code>==</code> / <code>hashCode</code>
Expecting int/double to interchange	<code>1</code> is int, <code>1.0</code> is double	Cast via <code>num: (j['x'] as num).toDouble()</code>

Part 8 • Troubleshooting

Symptom	Cause	Fix
<code>flutter</code> not recognised	PATH not applied	Open a new terminal; no spaces in the SDK path
<code>cmdline-tools</code> component is missing	SDK tool not installed	SDK Manager → SDK Tools → Command-line Tools (latest)
Android license status unknown, <code>-licenses</code> says "no longer needed"	Known Flutter bug with command-line tools 23.0+	Cosmetic - licences are accepted. Prove it with <code>flutter build apk --debug</code>
Android license status unknown (older tools)	Licences not accepted	<code>flutter doctor --android-licenses</code> → y to all
Multiple adb binaries found	platform-tools installed twice	Remove the duplicate; keep Android Studio's
Emulator stuck on the boot logo	No hardware acceleration	Hypervisor Driver / WHPX + BIOS virtualisation; else use a phone or Chrome
<code>adb devices</code> empty (Windows)	Missing USB driver or charge-only cable	Install the OEM driver; swap the cable
Unsupported class file major version	Wrong JDK	<code>flutter config --jdk-dir "C:\Program Files\Android\Android Studio\jbr"</code>
First build takes forever	It downloads ~4 GB: NDK, SDK platform, Gradle	Expected on a first build. Ask for the pre-warmed cache on the USB stick, or pair with someone who is already warmed
MissingPluginException	Hot reload after adding a native plugin	Stop and <code>flutter run</code> again
Hot reload "did nothing"	Change in <code>main()</code> / initialiser / <code>const</code> / enum	Press R (hot restart)
<code>setState()</code> called after <code>dispose()</code>	Async callback after leaving the screen	<code>if (!mounted) return;</code> after every <code>await</code>
Filename too long (git, Windows)	Long paths off	<code>git config --system core.longpaths true</code>
Build broken for no reason	Stale artefacts	<code>flutter clean</code> then <code>flutter pub get</code>

Part 9 • Homework for Day 2

Read <https://docs.flutter.dev/ui/layout/constraints> - the whole page, including the 29 examples. About 25 minutes. Run a few of them in DartPad.

Be ready to answer these at the whiteboard at 09:00 tomorrow:

1. Say the rule "*constraints go down, sizes go up, parent sets position*" in your own words, with one example.
2. Why does a `Container` with no child fill its parent, but the same `Container` inside a `Column` have zero height?
3. What is an "unbounded constraint" and which two widgets most often cause one?

4. Pick one of the 29 examples that surprised you and be ready to show it.

Layout is the biggest gap in the pre-course survey, so this reading is the preparation that makes tomorrow work.

Links from today

Topic	Link
Install (Windows)	https://docs.flutter.dev/get-started/install/windows
Install (macOS)	https://docs.flutter.dev/get-started/install/macos
Emulator acceleration	https://developer.android.com/studio/run/emulator-acceleration
Flutter architecture	https://docs.flutter.dev/resources/architectural-overview
Dart language tour	https://dart.dev/language
Null safety in depth	https://dart.dev/null-safety/understanding-null-safety
Patterns & sealed classes	https://dart.dev/language/patterns
DartPad	https://dartpad.dev
Widgets intro	https://docs.flutter.dev/ui/widgets-intro
Inside Flutter (three trees)	https://docs.flutter.dev/resources/inside-flutter
DevTools	https://docs.flutter.dev/tools/devtools
Packages	https://pub.dev
Tomorrow's reading	https://docs.flutter.dev/ui/layout/constraints